

Final Project Report

Team SocialDrinkers (3)

- [Final Project Report](#)
 - [Sprint 0 Summary: Requirement Gathering](#)
 - [Weekly Summary](#)
 - [Work Completed](#)
 - [Key Takeaways](#)
 - [Sprint 1 Summary: Design](#)
 - [Weekly Summary](#)
 - [Work Completed](#)
 - [Key Takeaways](#)
 - [Sprint 2 Summary: Low-Level Design](#)
 - [Weekly Summary](#)
 - [Work Completed](#)
 - [Key Takeaways](#)
 - [Sprint 3 Summary: Development](#)
 - [Weekly Summary](#)
 - [Work Completed](#)
 - [Incomplete Work](#)
 - [Key Takeaways](#)
 - [Sprint 4 Summary: Development](#)
 - [Weekly Summary](#)
 - [Work Completed](#)
 - [Incomplete / Ongoing Work](#)
 - [Key Takeaways](#)
 - [Sprint 5 Summary: Testing and Deployment](#)
 - [Weekly Summary](#)
 - [Work Completed](#)
 - [Incomplete / Ongoing Work](#)
 - [Key Takeaways](#)
 - [Project Documentation](#)
-

Sprint 0 Summary: Requirement Gathering

Team Lead: Dillan

Sprint 0 was dedicated to the initial conceptualization and foundational planning of the Codepop project. During this period, the team focused on defining the core project scope, gathering business and technical requirements, and establishing the standards for collaboration. This sprint was critical in transitioning our ideas into a structured roadmap, ensuring that every team member had a clear understanding of the project's decentralized architecture and the various user roles within the ecosystem.

Weekly Summary

- **Ethan H.** served as the primary documentarian and repository manager. He structured the initial project documentation, created the Use Case Diagrams for all system roles, and established the backend testing protocols to ensure future code quality.
 - **Ethan T.** focused on defining the core user-facing requirements, including the integrated payment system (Stripe), the logic for the AI drink recommendation engine, and ensuring cross-platform compatibility across mobile and web environments.
 - **Nate** established the team's technical workflow by authoring the Git Usage Standards and Code Standards. He also managed the initial repository configuration, ensuring that the team had a consistent set of guidelines for branching, commits, and code formatting from the very start.
 - **Dillan** led the sprint as Team Lead, overseeing the initial requirement gathering phase. He was responsible for drafting the project's high-level business goals and implementing the MoSCoW prioritization framework to guide the project's scope.
 - **Michael** developed the comprehensive suite of User Stories that formed the basis for the system's functional requirements. He defined the specific access levels and dashboard requirements for each of the seven identified user roles, from Guest Customers to Super Admins.
-

Work Completed

- **Project Requirements:** Finalized a comprehensive **Requirements.md** document covering business goals, functional requirements (Must/Should/Could Have), and detailed user stories.
 - **System Architecture Design:** Defined the decentralized, peer-to-peer network structure and regional supply hub model.
 - **Use Case Modeling:** Created detailed Use Case Diagrams for Customer Experience, Store Management, and Logistics Management to visualize system interactions.
 - **Technical Standards:** Established Git usage protocols, branching strategies, and initial code formatting standards.
 - **Prioritization Framework:** Implemented a MoSCoW plan to define the MVP (minimum viable product) and future feature roadmap.
 - **Repository Setup:** Configured the initial repository structure, **.gitignore** rules, and environment management tools.
-

Key Takeaways

The primary takeaway from Sprint 0 was the successful transition from abstract concepts to a concrete, documented roadmap. By establishing detailed User Stories and Use Case Diagrams early on, the team ensured a unified vision for the product's scope and stakeholder needs. Furthermore, the early adoption of Git and Code Standards laid a critical foundation for conflict-free collaboration, while the clear commitment to a decentralized P2P architecture provided the technical clarity necessary to prevent architectural drift as the project progressed into more complex development phases.

Sprint 1 Summary: Design

Team Lead: Nate Washburn

Sprint 1 was focused on the High-Level Design of the Codepop system. Building on the requirements and user stories established in Sprint 0, this sprint moved the project from conceptual scope into a concrete architectural direction. The team committed to a technology stack, defined the overall system structure, and produced a complete High-Level Design Document along with the first iteration of the UI prototype. This sprint was essential in giving the team a shared technical vocabulary and a clear picture of how the system would be built before any implementation work began.

Weekly Summary

- **Nate** served as Team Lead for the sprint and drove the UI direction by producing the Figma prototype. He also handled the component and interface modeling and helped shape the decentralized architecture, ensuring the design could scale toward a peer-to-peer model in later sprints.
 - **Dillan** took ownership of the security and encryption policies, contributed to the data classification matrix, and led the UML diagram work in coordination with Michael. He also supported the overall document review and refinement process.
 - **Ethan H.** led the initial drafting of the architecture narrative and the external interface documentation. He also contributed to the data classification matrix and was the primary contributor to producing the first full skeleton of the High-Level Design Document.
 - **Ethan T.** focused on the decentralization model and internal interface planning, working to outline how store-based routing would function. He also contributed to refining the security section of the document.
 - **Michael** defined the hardware platform strategy and documented the web-first Progressive Web App approach. He also contributed significantly to the UML diagrams and supported the component modeling effort.
-

Work Completed

- **Technology Stack Selection:** Finalized the stack as React with Next.js for the frontend, Django for the backend, PostgreSQL for the database, and Stripe for payment processing.
- **Hardware Platform Strategy:** Committed to a web-first approach using a Progressive Web App, allowing the system to run across desktop and mobile without maintaining separate codebases.
- **System Architecture:** Drafted the full architecture narrative, including the breakdown of major system components and their responsibilities.
- **Decentralization Model:** Established the structural approach for a decentralized network based on store selection, with multi-server aggregation reserved for admin-level users.
- **Interface Definition:** Documented both internal and external interfaces to clarify how system components and third-party services would interact.
- **Security & Data Classification:** Produced a data classification matrix and documented the encryption and security policies that would guide all future development.
- **UML Diagrams:** Completed the Class Diagram, Sequence Diagram, and Use Case Diagram to visualize system structure and behavior.
- **UI Prototype:** Built the first functional Figma prototype, establishing the visual language, color palette, and core user flows.
- **High-Level Design Document:** Finalized the complete HLD, consolidating all of the above into a single authoritative reference for the team.

Key Takeaways

The biggest win of Sprint 1 was reaching full team alignment on the technology stack and overall system direction. The decision to replace the existing frontend with React and Next.js was made deliberately and with consensus, and the team found that the long-term benefits of maintainability, PWA support, and development velocity outweighed the short-term cost of starting over. The UI prototyping work also turned out to be more valuable than expected, because it gave the team something concrete to anchor abstract architectural discussions around. By the end of the sprint, the team had a shared vocabulary, a documented plan, and a visual target to build toward, which set a strong foundation for the detailed design work that would follow in Sprint 2.

Sprint 2 Summary: Low-Level Design

Team Lead: Michael Seeley

Sprint 2 focused on transitioning from high-level architectural concepts to detailed low-level design specifications. The primary objective was to produce a comprehensive Low-Level Design (LLD) document that would serve as the technical blueprint for the upcoming development phase. This included defining the database schema, detailing subsystem interactions, designing high-fidelity UI prototypes, and formalizing the peer-to-peer (P2P) networking logic. Additionally, the team finalized the High-Level Design (HLD) to ensure a stable architectural foundation before proceeding with granular implementation details.

Weekly Summary

- **Michael** served as Team Lead and was responsible for the architectural overview and introduction of the LLD. He developed the detailed UML Class Diagrams, illustrating the relationships between core system entities such as Users, Drinks, and Orders. He also oversaw the finalization of the High-Level Design document and prepared technical diagrams for the mid-sprint "Show and Tell" presentation.
- **Ethan H.** acted as the primary architect for the LLD document structure. He established the document template, authored a detailed Table of Contents, and was the lead contributor to the Database Design section, where he defined the PostgreSQL schema and normalization strategies. He also managed repository maintenance, including the implementation of automated code formatting for documentation and `.gitignore` optimizations.
- **Ethan T.** focused on user-facing design and external system integrations. He authored the UI/UX Design section of the LLD and produced a comprehensive suite of prototypes for all user roles, including Customers, Managers, and Admins. Additionally, he documented the third-party integration strategies for critical services including Stripe (payments), OpenAI (AI recommendations), and Google Maps (location services).
- **Dillan** took ownership of the complex Peer-to-Peer (P2P) networking specifications. He defined the logic for Home and Visiting server assignments, cross-server session handling, and data synchronization protocols. He also contributed to the logistics management design and performed rigorous quality assurance on the LLD to eliminate inconsistencies and ensure technical accuracy across all sections.

- **Nate** collaborated on the development of the P2P networking model, focusing on fault tolerance and scalability aspects of the decentralized architecture. He also supported the refinement of the component and interface models, ensuring alignment with the security and performance goals established during the earlier design phases.
-

Work Completed

- **Low-Level Design Document:** Authored a comprehensive technical specification covering all nine major system subsystems and their interactions.
 - **Database Schema:** Finalized a fully normalized PostgreSQL schema, complete with detailed table definitions, foreign key relationships, and indexing strategies.
 - **UI/UX Prototypes:** Developed high-fidelity wireframes and flow diagrams for seven distinct user roles, establishing the visual language and core user journeys.
 - **P2P Networking Logic:** Formalized the "Home vs. Visiting Server" architecture, including authentication handshakes, data ownership rules, and synchronization triggers.
 - **UML Modeling:** Completed detailed Class and Sequence diagrams to map internal logic and facilitate inter-subsystem communication.
 - **Third-Party Integration Plans:** Documented technical implementation details and API security protocols for Stripe, OpenAI, and Google Maps.
 - **High-Level Design Finalization:** Integrated stakeholder feedback into the HLD to create a finalized baseline for the project's architecture.
-

Key Takeaways

The primary success of Sprint 2 was the creation of a "developer-ready" blueprint that significantly reduced ambiguity for the subsequent implementation phases. By investing heavily in the P2P networking and database design early, the team was able to identify and resolve potential data integrity challenges before any application code was written. The mid-sprint "Show and Tell" was a critical milestone, allowing the team to validate UI/UX flows and ensure the system's complexity remained manageable for all stakeholders. While the decentralized networking logic proved to be the most challenging aspect of the design, the collaborative effort between the architectural and networking leads resulted in a robust model that balances local autonomy with network-wide data consistency.

Sprint 3 Summary: Development

Team Lead: Ethan Huff

During Sprint 3 the team transitioned from design to active development. The primary objective was to establish a functional MVP by splitting the team into dedicated frontend and backend development sub-teams. This approach allowed for parallel development and specialized focus into each aspect of the project. To ensure efficiency and prevent duplication of efforts, we implemented a decentralized management style:

- **Task Management:** Major goals were decomposed into granular tasks and tracked as GitHub Issues. Team Members self-assigned the tasks between each other based on their expertise and availability.
- **Code Quality:** We established a mandatory peer-review process. Branch protection was enabled, preventing code from being pushed directly to the master branch until it had been reviewed by at least one other team member, preferably someone within the same sub-team.

- **Documentation:** Backend endpoints were documented using Swagger to provide a comprehensive view of the API endpoints and their functionalities.

Weekly Summary

Ethan H. finalized the Django models to align with the Low-Level Design, developed the core REST API endpoints (including the backend JWT authentication), and integrated Swagger documentation. He also implemented the Stripe payment API and created a comprehensive Integration Testing Suite to simulate user traffic.

Ethan T. managed the transition of UI/UX designs from Figma into functional ReactJS components and focused on the unregistered customer flow, ensuring we had a basic layout to work upon.

Michael developed the Store Manager Dashboard interface and began work on the Logistics Manager views. He also began integrating these frontend views with the newly created backend endpoints.

Nate handled the initial frontend repository setup and project structure. He successfully implemented the robust Frontend Authentication system, including the logic for handling JWT and Refresh tokens to maintain user sessions across page reloads.

Dillan spent substantial time focusing on backend infrastructure, specifically the P2P database syncing logic to ensure data integrity across the network. He also led the initial effort to containerize the application using Docker.

Work Completed

- **Authentication & Security:** Established a JWT-based auth system using Asymmetric RSA keys. This includes registration, login, and token refresh logic.
- **Core Backend API:** Developed the RESTful API required for the frontend to interact with the database, fully documented via Swagger.
- **Frontend Foundation:** Migrated Figma prototypes to a working React environment. Completed the initial functionality of the Home, Drink Customization, and Cart pages.
- **Database Infrastructure:** Django models were fully implemented and verified against design specs. Initial work on P2P synchronization and Docker containerization was completed.
- **Payment Integration:** Initial integration with the Stripe API for order processing and payment verification.
- **Testing Suite:** Created an Integration Testing Suite that manages headers and cookies to accurately test API responses under authenticated conditions.

Incomplete Work

There still remained certain aspects that, although were goals for this sprint, had to be deferred for future sprints due to time and complexity constraints, including:

- **P2P Network Synchronization:** The logic for background syncing of the master list across the network is largely complete but requires additional edge-case testing before being merged into the master branch.

- **Stripe Webhooks & Security:** While the payment portal is functional, the server-side webhooks for payment verification and the migration of API keys to environment variables were deferred to the next sprint.
 - **Managerial Dashboards:** The Store Manager and Logistics Manager views were partially implemented. The team decided to prioritize the completion of the "Customer Flow" (ordering and payment) before finalizing these internal employee tools.
 - **Containerization Finalization:** Initial Docker configurations were successful, but final deployment testing across different hardware environments is still pending.
-

Key Takeaways

The transition to a sub-team structure was highly effective. By utilizing Swagger for API documentation early in the sprint, the frontend and backend teams were able to work synchronously with minimal friction. We found that our modular design significantly reduced merge conflicts, allowing us to focus on development rather than version control issues. A major highlight was the Integration Testing Suite, which provided a reliable method for verifying our JWT authentication logic in a simulated environment.

Sprint 4 Summary: Development

Team Lead: Ethan Tatton

Weekly Summary

The team made strong progress across backend, frontend, and infrastructure, with clear ownership of tasks and consistent collaboration.

Ethan H. focused heavily on backend development and infrastructure. He also worked extensively on API development, server setup, and database stability, while maintaining scalability and organization across the codebase.

Ethan T. primarily handled frontend development. He redesigned the front page, implemented API integrations, and contributed to building out the admin interface.

Michael concentrated on the manager pages and their supporting API endpoints. He collaborated closely with backend work, ensuring functionality aligned between systems, and contributed to debugging and problem-solving.

Nate worked across both frontend and backend. He reorganized the project structure for better scalability, implemented authentication with multiple access levels, contributed to API development, and led development of the admin pages. He also supported debugging and integration efforts.

Dillan focused on backend systems, testing, and infrastructure. He played a major role in Dockerization, created automation scripts for server setup and testing, and worked on peer-to-peer networking and server synchronization. He also developed logistic manager pages, API endpoints, and geolocation functionality.

Work Completed

The team completed a wide range of development tasks, including:

- Full implementation of registered and unregistered user functionality
 - Backend API development and integration with frontend features
 - Migration to Docker for consistent development environments
 - Refactoring and restructuring the codebase for scalability
 - Implementation of authentication and user role management
 - Development of admin, manager, and logistic manager pages (partially complete)
 - AI-based drink recommendation system using user data
 - Peer-to-peer server architecture with synchronization
 - Backend testing systems and automation scripts
 - Geolocation and frontend enhancements (social features, settings, checkout)
-

Incomplete / Ongoing Work

Some features were started but not fully completed:

- Leaderboard (started late)
- Admin page features (partially implemented)
- Manager page frontend-backend integration issues
- AI chatbot (backend partially done, frontend incomplete)

These tasks are planned to be finalized in the next sprint, which will focus on testing and stabilization.

Key Takeaways

- The team demonstrated strong collaboration, especially through asynchronous communication
- Major technical milestones (Docker migration, backend stability, API completion) were successfully achieved
- Some features were too large or started too late, leading to incomplete implementation
- Better task breakdown, earlier prioritization, and improved frontend-backend coordination are needed moving forward
- Next sprint will prioritize testing, debugging, and completing near-finished features

Sprint 5 Summary: Testing and Deployment

Team Lead: Dillan

Sprint 5 was the final development sprint, focused on testing, deployment, and stabilizing the application for the end-of-semester demo. The primary challenge of the sprint was standing up a true multi-store environment and verifying that all features worked correctly across simultaneously running instances. A significant portion of the sprint was spent recovering from broken frontend code that had accumulated over prior commits.

Weekly Summary

Ethan H. was a primary driver of backend feature completion this sprint. He built the global revenue aggregation view for super admins, automated revenue record generation tied to order completion, and

fixed order total calculation bugs. He implemented price calculation logic for drink size and add-on charges, repaired the inventory restocking flow, and added home server details and role editing to user profiles. He also did two rounds of fixes on Stripe webhook handling, audited all frontend server fetch calls to route through the proxy, and enhanced post-order confirmation with JIT tracking logic. He contributed to the User Manual and the Test Design Report documentation.

Ethan T. focused primarily on documentation and testing. He updated existing design documents based on feedback received and revised them to more closely reflect the features the team had actually implemented. He reviewed and edited the User Manual and assisted with bug testing when brought in directly during the final stretch of the sprint.

Michael contributed to the Test Design Report and reviewing/editing the User Manual. He also performed some software testing.

Nate handled frontend UI work throughout the sprint. He reverted several broken frontend commits to restore a clean build state, updated the sidebar with correct navigation and role-based permissions, refreshed background colors across admin and user pages for visual consistency, added a new app icon, and merged in the social pages feature. He also started the User Manual and filled out the majority of its content.

Dillan led the multi-store infrastructure effort. He created a **docker - compose** file that launches 9 simultaneous store instances, fixed all inter-instance routing through port 4000, and implemented P2P network info discovery. He built a server switcher UI and a locations page, added randomized seed data per store, and spent multiple days tracking down and fixing multi-store bugs. He also fixed session persistence so users stay signed in when switching stores and implemented proxying of store manager data across servers so a manager signed into any store can access their home store's information.

Work Completed

- **Multi-Store Infrastructure:** Launched 9 simultaneous store instances via Docker Compose, with working inter-instance routing, P2P discovery, a server switcher UI, and randomized per-store seed data.
- **Session Persistence:** Fixed sign-in persistence across store switches and implemented cross-store authentication proxying for store managers.
- **Revenue System:** Built a global revenue aggregation view for super admins with automated revenue record generation tied to order completion and accurate reporting alongside inventory changes.
- **Stripe Webhooks:** Completed two rounds of fixes to stabilize webhook handling for payment intents and revenue records.
- **Inventory System:** Repaired the broken restocking flow and improved stock-level reporting accuracy.
- **Drink Pricing:** Implemented price calculation logic accounting for drink size and add-on charges.
- **User Management:** Added home server details to profiles, implemented admin role editing, and secured the profile editing endpoint.
- **JIT Order Tracking:** Enhanced post-order confirmation flow with Just-In-Time tracking and estimated arrival logic.
- **API Proxy Consistency:** Audited and updated all frontend server fetch calls to route through the proxy, resolving cross-origin issues.
- **Frontend UI & Sidebar:** Restored clean build state from broken commits, updated sidebar navigation with role-based permissions, refreshed page backgrounds, added a new app icon, and merged social

pages.

- **Leaderboard Backend:** Merged the backend leaderboard view.
 - **Documentation:** Completed the User Manual, updated the Test Design Report, and revised existing design documents to reflect the implemented system.
-

Incomplete / Ongoing Work

- **Logistics Manager:** The scaffolding was merged and some UI was built, but the backend logic for transferring stock between stores was never fully implemented and the feature is not functional.
 - **Multi-Store on Google Cloud:** Deployment to Google Cloud was attempted but took too long and was abandoned in favor of a local Docker-based demo.
 - **Repair Staff View:** This feature was cut entirely and never started.
-

Key Takeaways

- Individual components that work separately often break in weird and unsuspecting ways when brought together to make a larger piece of software.
 - The authentication proxying system for multi-store sessions was architecturally sound but was only tested on the super admin account, whose elevated permissions masked a real bug in the manager flow. Testing across multiple role types from the start would have caught this earlier.
-

Project Documentation

This section contains links to the various design and project documents created throughout the lifecycle of the Codepop project.

- **Requirements Document:** [Requirements.md](#)
- **High-Level Design (HLD):** [HighLevelDesign.md](#)
- **Low-Level Design (LLD):** [LowLevelDesign.md](#)
- **Test Design Report:** [TestDesignReport.md](#)
- **Project Presentation:** [CodePop - Presentation.pdf](#)